Quantum Computing, Quantum Machine Learning and Quantum Information Theories

Morten Hjorth-Jensen[1]

Department of Physics, University of Oslo, Norway[1]

April 2, 2025

Plans for the week of March 31-April 4, 2025

1. Discrete Fourier transforms (DFTs, reminder from last week)) and the fast Fourier Transform (FFT) (additional slides)
2. Quantum Fourier transforms (QFTs), reminder from last week
3. Setting up circuits for QFT
4. Quantum phase estimation algorithm
5. Reading recommendation Hundt, Quantum Computing for Programmers, sections 6.1-6.4 on QFT.

Brief reminder on Fourier transforms

Last week we reviewed some basic properties of Fourier transforms and derived the expressions for the quantum Fourier variant. We restate some of these results here. For more information, see slides from previous week

Discrete Fourier transforms

The Discrete Fourier Transform is a tool used in many different fields and finds applications in for example signal processing. It has many many applications (in Speech Recognition, Data Processing, Optics, Acoustics). It is used in any application which performs Digital Signal Processing.
There is an efficient algorithm, called the Fast Fourier Transform (FFT), which uses the special relationship amongst the roots of unity to compute the entire DFT in $O(n \log n)$ time. Moreover, it is possible to take the DFT and recover the original polynomial in its coefficient form in $O(n \log n)$ time, by using the FFT to compute the inverse DFT.

Fast Fourier transform (FFT)

Because of the many important applications of DFT, the Fast Fourier Transform is ranked among the top ten algorithms in the 20th century. It came to light as a tool for Signal processing in 1965, when Cooley (of IBM) and Tukey (of Princeton) wrote a paper presenting the algorithm. However the basics of the algorithm were known long before that, for example, it was known to Gauss back in 1805. To read more about Fast Fourier transforms and similar topics, see for example Fast Fourier Transform - Algorithms and Applications. See also `https://github.com/CompPhysics/QuantumComputingMachineLearning/blob/gh-pages/doc/Textbooks/fastfourier.pdf`
For a discussion of FFT, see additional slides at (address to be added)

The discrete Fourier transform (DFT)

The discrete Fourier transform takes as input a complex vector

$$\boldsymbol{x} = |\boldsymbol{x}\rangle = \begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ \vdots \\ x_{n-2} \\ x_{n-1} \end{bmatrix}.$$

Each entry of the output vector $|\boldsymbol{y}\rangle$ is given by

$$y_k = \frac{1}{\sqrt{N}} \sum_{j=0}^{N-1} x_j e^{2\pi i j k / N}.$$

Output vector

The output is a complex vector

$$|\boldsymbol{y}\rangle = \frac{1}{\sqrt{n}} \begin{bmatrix} \sum_{j=0}^{n-1} x_j e^{2\pi i j 0/n} \\ \sum_{j=0}^{n-1} x_j e^{2\pi i j 1/n} \\ \sum_{j=0}^{n-1} x_j e^{2\pi i j 2/n} \\ \vdots \\ \sum_{j=0}^{n-1} x_j e^{2\pi i j (n-2)/n} \\ \sum_{j=0}^{n-1} x_j e^{2\pi i j (n-1)/n} \end{bmatrix}$$

Simple example

As an example we can have a set of complex numbers $\{x_0, \ldots, x_{n-1}\}$ with fixed length n, we can Fourier transform this as

$$y_k = \frac{1}{\sqrt{n}} \sum_{j=0}^{n-1} x_j \exp i(2\pi jk)/n,$$

leading to a new set of complex numbers $\{y_0, \ldots, y_{n-1}\}$.

Discrete Fourier Transformations

Consider two sets of complex numbers x_k and y_k with $k = 0, 1, \ldots, n-1$ entries. The discrete Fourier transform is defined as

$$y_k = \frac{1}{\sqrt{n}} \sum_{j=0}^{n-1} \exp\left(\frac{2\pi \imath jk}{n}\right) x_j.$$

As an example, assume $x_0 = 1$ and $x_1 = 1$. We can then use the above expression to find y_0 and y_1.
With the above formula we get then

$$\begin{aligned} y_0 &= \frac{1}{\sqrt{2}} \left(\exp\left(\frac{2\pi \imath 0 \times 1}{2}\right) \times 1 + \exp\left(\frac{2\pi \imath 0 \times 1}{2}\right) \times 2 \right) \\ &= \frac{1}{\sqrt{2}}(1+2) = \frac{3}{\sqrt{2}}, \end{aligned}$$

and

$$\begin{aligned} y_1 &= \frac{1}{\sqrt{2}} \left(\exp\left(\frac{2\pi \imath 0 \times 1}{2}\right) \times 1 + \exp\left(\frac{2\pi \imath 1 \times 1}{2}\right) \times 2 \right) = \\ &= \frac{1}{\sqrt{2}}(1 + 2\exp(\pi \imath)) = -\frac{1}{\sqrt{2}}. \end{aligned}$$

More details on Discrete Fourier transforms

Suppose that we have a vector f of n complex numbers, $f_k, k \in \{0, 1, \ldots, n-1\}$. Then the discrete Fourier transform (DFT) is a map from these n complex numbers to n complex numbers, the Fourier transformed coefficients $\tilde{f}_j$, given by

$$\tilde{f}_j = \frac{1}{\sqrt{n}} \sum_{k=0}^{n-1} \omega^{-jk} f_k$$

where $\omega = \exp\left(\frac{2\pi i}{N}\right)$.

Inverse DFT

The inverse DFT is given by

$$f_j = \frac{1}{\sqrt{n}} \sum_{k=0}^{n-1} \omega^{jk} \tilde{f}_k$$

To see this consider how the basis vectors transform. If $f_k^l = \delta_{k,l}$, then

$$\tilde{f}_j^l = \frac{1}{\sqrt{n}} \sum_{k=0}^{n-1} \omega^{-jk} \delta_{k,l} = \frac{1}{\sqrt{n}} \omega^{-jl}$$

Orthonormality

These DFT vectors are orthonormal, that is

$$\sum_{j=0}^{n-1} \tilde{f}^{l*}_j \tilde{f}^m_j = \frac{1}{n} \sum_{j=0}^{n-1} \omega^{jl} \omega^{-jm} = \frac{1}{N} \sum_{j=0}^{n-1} \omega^{j(l-m)}$$

This last sum can be evaluated as a geometric series, but beware of the $(l - m) = 0$ term, and yields

$$\sum_{j=0}^{n-1} \tilde{f}^{l*}_j \tilde{f}^m_j = \delta_{l,m}$$

Inverse transform

From this we can check that the inverse DFT does indeed perform the inverse transform:

$$f_j = \frac{1}{\sqrt{n}} \sum_{k=0}^{n-1} \omega^{jk} \tilde{f}_k = \frac{1}{\sqrt{n}} \sum_{k=0}^{n-1} \omega^{jk} \frac{1}{\sqrt{n}} \sum_{l=0}^{n-1} \omega^{-lk} f_l = \frac{1}{n} \sum_{k,l=0}^{n-1} \omega^{(j-l)k} f_l = \sum_{l=0}^{n-1} \delta$$

From DFT to QFT

The Quantum Fourier Transform (QFT) has mathematically the same equation as starting point but the notation is generally different. We generally compute the quantum Fourier transform on a set of orthonormal basis state vectors $|0\rangle, |1\rangle, ..., |N-1\rangle$. The linear operator defining the transform is given by the action on basis states

$$|j\rangle \mapsto \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} e^{2\pi ijk/N} |k\rangle.$$

In terms of arbitrary states

This can be written on arbitrary states,

$$\sum_{j=0}^{N-1} x_j |j\rangle \mapsto \sum_{k=0}^{N-1} y_k |k\rangle$$

where each amplitude y_k is the discrete Fourier transform of x_j.

Unitarity

The quantum Fourier transform is unitary. Taking $N = 2^n$, for n qubits gives us the orthonormal (computational) basis

$$|0\rangle, |1\rangle, ..., |2^n - 1\rangle.$$

Binary representation

Each of the computational basis states can be represented in binary

$$j = j_1 j_2 \cdots j_n$$

where each j_k is either 0 or 1, and the corresponding binary vector is $|j_1 j_2 \cdots j_n\rangle$.

Rewriting the QFT

The quantum Fourier transform on one of these n-qubit vectors can be written as,

$$|j_1 j_2 \cdots j_n\rangle = \frac{\left(|0\rangle + e^{2\pi i 0.j_n}|1\rangle\right) \otimes \left(|0\rangle + e^{2\pi i 0.j_{n-1}j_n}|1\rangle\right) \otimes \cdots \otimes \left(|0\rangle + e^{2\pi}\right.}{2^{n/2}}$$

Short hand notation

In the above, we use the notation

$$0.j_l j_{l+1} \cdots j_n = \frac{j+l}{2} + \frac{j_{l+1}}{2^2} + \cdots + \frac{j_n}{2^{m-l+1}}$$

Two-qubit system

First a two qubit system. See whiteboard notes.

Four qubit system

Let us then move to a four qubit system. The basis states are,

$$|j_1 j_2 j_3 j_4\rangle$$

where j_k is either 0 or 1. We have

$$\begin{aligned}
0.j_3 &= \frac{j_3}{2} \\
0.j_2 j_3 &= \frac{j_2}{2} + \frac{j_3}{4} \\
0.j_1 j_2 j_3 &= \frac{j_1}{2} + \frac{j_2}{4} + \frac{j_3}{8} \\
0.j_0 j_1 j_2 j_3 &= \frac{j_0}{2} + \frac{j_1}{4} + \frac{j_2}{8} + \frac{j_3}{16}
\end{aligned}$$

QFT acts as follows

The quantum Fourier transform acts as follows:

$$|j_1 j_2 j_3 j_4\rangle \mapsto \frac{1}{\sqrt{2^{4/2}}} \left(|0\rangle + e^{2\pi i 0.j4}|1\rangle\right) \otimes \left(|0\rangle + e^{2\pi i 0.j_3 j4}|1\rangle\right) \otimes \left(|0\rangle + e^{2\pi i 0}\right.$$

Quantum circuits

To compose a quantum circuit that calculates the quantum Fourier transform we use the operators

$$R_k = \begin{bmatrix} 1 & 0 \\ 0 & e^{2\pi i/2^k} \end{bmatrix}.$$

For the circuit computing the QFT on four qubits, see whiteboard notes again.

Rotation gates

In this example, the R_k gates are:

$$R_1 = \begin{bmatrix} 1 & 0 \\ 0 & e^{2\pi i/2^0} \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \quad R_2 = \begin{bmatrix} 1 & 0 \\ 0 & e^{2\pi i/2^2} \end{bmatrix}, \quad R_3 = \begin{bmatrix} 1 & 0 \\ 0 & e^{2\pi i/2^3} \end{bmatrix}$$

Quantum Fourier transform, more material

We have defined the QFT in terms of the unitary operation

$$|\psi'\rangle \leftarrow \hat{F}|\psi\rangle, \quad \hat{F}^\dagger \hat{F} = I$$

Orthonormal basis

In terms of an orthonormal basis $|0\rangle, |1\rangle, \ldots, |0\rangle$ this linear operator has the following action

$$|j\rangle \rightarrow \sum_{k=0}^{N-1} \exp i(2\pi jk/N)|k$$

or on an arbitrary state

$$\sum_{j=0}^{N-1} x_j|j\rangle \rightarrow \sum_{k=0}^{N-1} y_k|k\rangle$$

equivalent to the equation for discrete Fourier transform on a set of complex numbers.

Using computational basis

Next we assume an n-qubit system, where we take $N = s^n$ in the computational basis

$$|0\rangle, \ldots, |2^n - 1\rangle.$$

We make use of the binary representation $j = j_1 2^{n-1} + j_2 2^{n-2} + \cdots + j_n 2^0$, and take note of the notation $0.j_l j_{l+1} \ldots j_m$ representing the binary fraction $\frac{j_l}{2^1} + \frac{j_{l+1}}{2^2} + \cdots + \frac{j_m}{2^{m-l+1}}$. With this we define the product representation of the quantum Fourier transform

$$|j_1, \ldots, j_n\rangle \rightarrow \frac{(|0\rangle + \exp i(2\pi 0.j_n))\,(|0\rangle + \exp i(2\pi 0.j_{j-1} j_n)) \ldots (|0\rangle + \exp$$
$$2^{n/2}$$

Components

From the product representation we can derive a circuit for the quantum Fourier transform. This will make use of the following two single-qubit gates

$$H = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$$

$$R_k = \begin{bmatrix} 1 & 0 \\ 0 & e^{2\pi i/2^k} \end{bmatrix}$$

Using the Hadamard gate

The Hadamard gate on a single qubit creates an equal superposition of its basis states, assuming it is not already in a superposition, such that

$$H|0\rangle = \frac{1}{\sqrt{2}}\left(|0\rangle + |1\rangle\right), \quad H|1\rangle = \frac{1}{\sqrt{2}}\left(|0\rangle - |1\rangle\right)$$

The R_k gate simply adds a phase if the qubit it acts on is in the state $|1\rangle$

$$R_k|0\rangle = |0\rangle, \quad R_k|1\rangle = e^{2\pi i/2^k}|1\rangle$$

Since all this gates are unitary, the quantum Fourier transfrom is also unitary.

Algorithm

Assume we have a quantum register of n qubits in the state $|j_1 j_2 \dots j_n\rangle$. Applying the Hadamard gate to the first qubit produces the state

$$H|j_1 j_2 \dots j_n\rangle = \frac{\left(|0\rangle + e^{2\pi i 0.j_1}|1\rangle\right)}{2^{1/2}}|j_2 \dots j_n\rangle.$$

Binary fraction

Here we have made use of the binary fraction to represent the action of the Hadamard gate

$$\exp 2\pi i 0.j_1 = -1,$$

if $j_1 = 1$ and $+1$ if $j_1 = 0$.

Controlled rotation gate

Furthermore we can apply the controlled-R_k gate, with all the other qubits j_k for $k > 1$ as control qubits to produce the state

$$\frac{\left(|0\rangle + e^{2\pi i 0.j_1 j_2 \ldots j_n}|1\rangle\right)}{2^{1/2}}|j_2 \ldots j_n\rangle$$

Next we do the same procedure on qubit 2 producing the state

$$\frac{\left(|0\rangle + e^{2\pi i 0.j_1 j_2 \ldots j_n}|1\rangle\right)\left(|0\rangle + e^{2\pi i 0.j_2 \ldots j_n}|1\rangle\right)}{2^{2/2}}|j_2 \ldots j_n\rangle$$

Applying to all qubits

Doing this for all n qubits yields state

$$\frac{\left(|0\rangle + e^{2\pi i 0.j_1 j_2 \ldots j_n}|1\rangle\right)\left(|0\rangle + e^{2\pi i 0.j_2 \ldots j_n}|1\rangle\right)\ldots\left(|0\rangle + e^{2\pi i 0.j_n}|1\rangle\right)}{2^{n/2}}|j_2 \ldots j_n\rangle$$

At the end we use swap gates to reverse the order of the qubits

$$\frac{\left(|0\rangle + e^{2\pi i 0.j_n}|1\rangle\right)\left(|0\rangle + e^{2\pi i 0.j_{n-1} j_n}|1\rangle\right)\ldots\left(|0\rangle + e^{2\pi i 0.j_1 j_2 \ldots j_n}|1\rangle\right)}{2^{n/2}}|j_2 \ldots j_n\rangle$$

This is just the product representation from earlier, obviously our desired output.

Quantum Fourier transform

Now lets turn to the Quantum Fourier transform (QFT). We've already seen the QFT for $N = 2$. It is the Hadamard transform:

$$H = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$$

QFT for $N = 2$

Why is this the QFT for $N = 2$? Well suppose have the single qubit state $a_0|0\rangle + a_1|1\rangle$. If we apply the Hadamard operation to this state we obtain the new state

$$\frac{1}{\sqrt{2}}(a_0 + a_1)|0\rangle + \frac{1}{\sqrt{2}}(a_0 - a_1)|1\rangle = \tilde{a}_0|0\rangle + \tilde{a}_1|1\rangle.$$

In other words the Hadamard gate performs the DFT for $N = 2$ on the amplitudes of the state! Notice that this is very different that computing the DFT for $N = 2$: remember the amplitudes are not numbers which are accessible to us mere mortals, they just represent our description of the quantum system.

Full QFT

So what is the full quantum Fourier transform? It is the transform which takes the amplitudes of a N dimensional state and computes the Fourier transform on these amplitudes (which are then the new amplitudes in the computational basis.) In other words, the QFT enacts the transform

$$\sum_{x=0}^{N-1} a_x|x\rangle \rightarrow \sum_{x=0}^{N-1} \tilde{a}_x|x\rangle = \sum_{x=0}^{N-1} \frac{1}{\sqrt{N}} \sum_{y=0}^{N-1} \omega_N^{-xy} a_y|x\rangle$$

Explicit transform

It is easy to see that this implies that the QFT performs the following transform on basis states:

$$|x\rangle \rightarrow \frac{1}{\sqrt{N}} \sum_{y=0}^{N-1} \omega_N^{-xy} |y\rangle$$

Thus the QFT is given by the matrix

$$U_{QFT} = \frac{1}{\sqrt{N}} \sum_{x=0}^{N-1} \sum_{y=0}^{N-1} \omega_N^{-yx} |y\rangle\langle x|$$

Unitarity

The last matrix is unitary. Let's check this:

$$
\begin{aligned}
U_{QFT}\,U_{QFT}^{\dagger} &= \frac{1}{N}\sum_{x=0}^{N-1}\sum_{y=0}^{N-1}\omega_N^{yx}|x\rangle\left\langle y\right|\sum_{x'=0}^{N-1}\sum_{y'=0}^{N-1}\omega_N^{-y'x'}\left|y'\right\rangle\langle x'| \\
&= \frac{1}{N}\sum_{x,y,x',y'=0}^{N-1}\omega_N^{yx-y'x'}\delta_{y,y'}|x\rangle\left\langle x'\right| = \frac{1}{N}\sum_{x,y,x'=0}^{N-1}\omega_N^{y(x-x')}\left|x\right\rangle \\
&= \sum_{x,x'=0}^{N-1}\delta_{x,x'}|x\rangle\langle x'| = I
\end{aligned}
$$

Importance of QFT

The QFT is a very important transform in quantum computing. It can be used for all sorts of cool tasks, including, as we shall see in Shor's algorithm. But before we can use it for quantum computing tasks, we should try to see if we can efficiently implement the QFT with a quantum circuit. Indeed we can and the reason we can is intimately related to the fast Fourier transform.

Circuit QFT

Let's derive a circuit for the QFT when $N = 2^n$. The QFT performs the transform

$$|x\rangle \rightarrow \frac{1}{\sqrt{2^n}} \sum_{y=0}^{2^n-1} \omega_N^{-xy} |y\rangle$$

Then we can expand out this sum

$$|x\rangle \rightarrow \frac{1}{\sqrt{2^n}} \sum_{y_1,y_2,\ldots,y_n \in \{0,1\}} \omega_N^{-x \sum_{k=1}^{n} 2^{n-k} y_k} |y_1, y_2, \ldots, y_n\rangle$$

Expanding the exponential

Expanding the exponential of a sum to a product of exponentials and collecting these terms in from the appropriate terms we can express this as

$$|x\rangle \rightarrow \frac{1}{\sqrt{2^n}} \sum_{y_1,y_2,\ldots,y_n \in \{0,1\}} \bigotimes_{k=1}^{n} \omega_N^{-x2^{n-k}y_k} |y_k\rangle$$

Rearranging

We can rearrange the sum and products as

$$|x\rangle \rightarrow \frac{1}{\sqrt{2^n}} \bigotimes_{k=1}^{n} \left[\sum_{y_k \in \{0,1\}} \omega_N^{-x 2^{n-k} y_k} |y_k\rangle \right]$$

Expanding this sum yields

$$|x\rangle \rightarrow \frac{1}{\sqrt{2^n}} \bigotimes_{k=1}^{n} \left[|0\rangle + \omega_N^{-x 2^{n-k}} |1\rangle \right]$$

Notation for binary fraction

But now notice that $\omega_N^{-x2^{n-k}}$ is not dependent on the higer order bits of x. It is convenient to adopt the following expression for a binary fraction:

$$0.x_l x_{l+1} \ldots x_n = \frac{x_l}{2} + \frac{x_{l+1}}{4} + \cdots + \frac{x_n}{2^{n-l+1}}$$

More notations

Then we can see that

$$|x\rangle \rightarrow \frac{1}{\sqrt{2^n}} \left[|0\rangle + e^{-2\pi i 0.x_n}|1\rangle\right] \otimes \left[|0\rangle + e^{-2\pi i 0.x_{n-1}x_n}|1\rangle\right] \otimes \cdots \otimes \left[|0\rangle + e^{-2\pi}\right.$$

This is a very useful form of the QFT for $N = 2^n$. Why? Because we see that only the last qubit depends on the the values of all the other input qubits and each further bit depends less and less on the input qubits. Further we note that $e^{-2\pi i 0.a}$ is either +1 or -1 , which reminds us of the Hadamard transform.

Deriving a circuit

So how do we use this to derive a circuit for the QFT over $N = 2^n$? Take the first qubit of $|x_1, \ldots, x_n\rangle$ and apply a Hadamard transform. This produces the transform

$$|x\rangle \to \frac{1}{\sqrt{2}} \left[|0\rangle + e^{-2\pi i 0 \cdot x_1}|1\rangle\right] \otimes |x_2, x_3, \ldots, x_n\rangle$$

Rotation gate

Now define the rotation gate

$$R_k = \begin{bmatrix} 1 & 0 \\ 0 & \exp\left(\frac{-2\pi i}{2^k}\right) \end{bmatrix} \tag{30}$$

If we now apply controlled R_2, R_3, etc. gates controlled on the appropriate bits this enacts the transform

$$|x\rangle \rightarrow \frac{1}{\sqrt{2}}\left[|0\rangle + e^{-2\pi i 0.x_1 x_2 \ldots x_n}|1\rangle\right] \otimes |x_2, x_3, \ldots, x_n\rangle$$

Proceeding

Thus we have reproduced the last term in the QFTed state. Of course now we can proceed to the second qubit, perform a Hadamard, and the appropriate controlled R_k gates and get the second to last qubit. Thus when we are finished we will have the transform

$$|x\rangle \rightarrow \frac{1}{\sqrt{2^n}} \left[|0\rangle + e^{-2\pi i 0.x_1 x_2 \cdots x_n}|1\rangle\right] \otimes \left[|0\rangle + e^{-2\pi i 0.x_1 x_2 \cdots x_{n-1}}|1\rangle\right] \otimes \cdots \otimes \left[|0\right.$$

Reversing the order of these qubits will then produce the QFT!

QFT

The Quantum Fourier Transform (QFT) has mathematically the same equation as starting point but the notation is generally different. We generally compute the quantum Fourier transform on a set of orthonormal basis state vectors $|0\rangle, |1\rangle, ..., |N-1\rangle$. The linear operator defining the transform is given by the action on basis states

$$|j\rangle \mapsto \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} e^{2\pi ijk/N} |k\rangle.$$

In terms of arbitrary states

This can be written on arbitrary states,

$$\sum_{j=0}^{N-1} x_j |j\rangle \mapsto \sum_{k=0}^{N-1} y_k |k\rangle$$

where each amplitude y_k is the discrete Fourier transform of x_j.

Unitarity

The quantum Fourier transform is unitary. Taking $N = 2^n$, for n qubits gives us the orthonormal (computational) basis

$$|0\rangle, |1\rangle, ..., |2^n - 1\rangle.$$

Binary representation

Each of the computational basis states can be represented in binary

$$j = j_1 j_2 \cdots j_n$$

where each j_k is either 0 or 1, and the corresponding binary vector is $|j_1 j_2 \cdots j_n\rangle$.

Rewriting the QFT

The quantum Fourier transform on one of these n-qubit vectors can be written as,

$$|j_1 j_2 \cdots j_n\rangle = \frac{\left(|0\rangle + e^{2\pi i 0.j_n}|1\rangle\right) \otimes \left(|0\rangle + e^{2\pi i 0.j_{n-1}j_n}|1\rangle\right) \otimes \cdots \otimes \left(|0\rangle + e^{2\pi}\right.}{2^{n/2}}$$

Short hand notation

In the above, we use the notation

$$0.j_l j_{l+1} \cdots j_n = \frac{j_l}{2} + \frac{j_{l+1}}{2^2} + \cdots + \frac{j_n}{2^{n-l+1}}$$

If we start counting from zero, that is we have instead

$$|j_0 j_1 \cdots j_{n-1}\rangle = \frac{\left(|0\rangle + e^{2\pi i 0.j_{n-1}}|1\rangle\right) \otimes \left(|0\rangle + e^{2\pi i 0.j_{n-2} j_n}|1\rangle\right) \otimes \cdots \otimes \left(|0\rangle +}{2^{n/2}}$$

we get

$$0.j_l j_{l+1} \cdots j_{n-1} = \frac{j_l}{2} + \frac{j_{l+1}}{2^2} + \cdots + \frac{j_{n-1}}{2^{n-l}}$$

Four qubit system

The basis states are

$$|j_1 j_2 j_3 j_4\rangle$$

where j_k is either 0 or 1. We have

$$\begin{aligned}
0.j_3 &= \frac{j_3}{2} \\
0.j_2 j_3 &= \frac{j_2}{2} + \frac{j_3}{4} \\
0.j_1 j_2 j_3 &= \frac{j_1}{2} + \frac{j_2}{4} + \frac{j_3}{8} \\
0.j_0 j_1 j_2 j_3 &= \frac{j_0}{2} + \frac{j_1}{4} + \frac{j_2}{8} + \frac{j_3}{16}
\end{aligned}$$

QFT acts as follows

The quantum Fourier transform acts as follows:

$$|j_1 j_2 j_3 j_4\rangle \mapsto \frac{1}{\sqrt{2^{4/2}}} \left(|0\rangle + e^{2\pi i 0.j4}|1\rangle\right) \otimes \left(|0\rangle + e^{2\pi i 0.j_3 j4}|1\rangle\right) \otimes \left(|0\rangle + e^{2\pi i 0}\right.$$

Quantum circuits

To compose a quantum circuit that calculates the quantum Fourier transform we use the operators

$$R_k = \begin{bmatrix} 1 & 0 \\ 0 & e^{2\pi i/2^k} \end{bmatrix}.$$

Rotation gates

In this example, the R_k gates are:

$$R_1 = \begin{bmatrix} 1 & 0 \\ 0 & e^{2\pi i/2^0} \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \quad R_2 = \begin{bmatrix} 1 & 0 \\ 0 & e^{2\pi i/2^2} \end{bmatrix}, \quad R_3 = \begin{bmatrix} 1 & 0 \\ 0 & e^{2\pi i/2^3} \end{bmatrix}$$

Using the Hadamard gate

The Hadamard gate on a single qubit creates an equal superposition of its basis states, assuming it is not already in a superposition, such that

$$H|0\rangle = \frac{1}{\sqrt{2}}\left(|0\rangle + |1\rangle\right), \quad H|1\rangle = \frac{1}{\sqrt{2}}\left(|0\rangle - |1\rangle\right)$$

The R_k gate simply adds a phase if the qubit it acts on is in the state $|1\rangle$

$$R_k|0\rangle = |0\rangle, \quad R_k|1\rangle = e^{2\pi i/2^k}|1\rangle$$

Since all this gates are unitary, the quantum Fourier transfrom is also unitary.

Algorithm

Assume we have a quantum register of n qubits in the state $|j_1 j_2 \dots j_n\rangle$. Applying the Hadamard gate to the first qubit produces the state

$$H|j_1 j_2 \dots j_n\rangle = \frac{\left(|0\rangle + e^{2\pi i 0.j_1}|1\rangle\right)}{2^{1/2}}|j_2 \dots j_n\rangle.$$

Binary fraction

Here we have made use of the binary fraction to represent the action of the Hadamard gate

$$\exp 2\pi i 0.j_1 = -1,$$

if $j_1 = 1$ and $+1$ if $j_1 = 0$.

Controlled rotation gate

Furthermore we can apply the controlled-R_k gate, with all the other qubits j_k for $k > 1$ as control qubits to produce the state

$$\frac{\left(|0\rangle + e^{2\pi i 0.j_1 j_2 \ldots j_n}|1\rangle\right)}{2^{1/2}}|j_2 \ldots j_n\rangle$$

Next we do the same procedure on qubit 2 producing the state

$$\frac{\left(|0\rangle + e^{2\pi i 0.j_1 j_2 \ldots j_n}|1\rangle\right)\left(|0\rangle + e^{2\pi i 0.j_2 \ldots j_n}|1\rangle\right)}{2^{2/2}}|j_2 \ldots j_n\rangle$$

Applying to all qubits

Doing this for all n qubits yields state

$$\frac{\left(|0\rangle + e^{2\pi i 0.j_1 j_2 \ldots j_n}|1\rangle\right)\left(|0\rangle + e^{2\pi i 0.j_2 \ldots j_n}|1\rangle\right)\ldots\left(|0\rangle + e^{2\pi i 0.j_n}|1\rangle\right)}{2^{n/2}}|j_2 \ldots j_n\rangle$$

At the end we use swap gates to reverse the order of the qubits

$$\frac{\left(|0\rangle + e^{2\pi i 0.j_n}|1\rangle\right)\left(|0\rangle + e^{2\pi i 0.j_{n-1} j_n}|1\rangle\right)\ldots\left(|0\rangle + e^{2\pi i 0.j_1 j_2 \ldots j_n}|1\rangle\right)}{2^{n/2}}|j_2 \ldots j_n\rangle$$

This is just the product representation from earlier, obviously our desired output.